Ira A. Fulton Schools of Engineering
Arizona State University

CSE 531: Distributed and Multiprocessor Operating Systems

CSE531 Client-Centric Consistency Written Report

Casey Tom

Directions

Use this learner template to write your report for the **Client-Centric Consistency Project**. Replace the placeholder text within each section with the required information. You may add subheadings on the template to purposefully call attention to specific, organized details. Your written report must be a single PDF with the correct naming convention: *CSE 531_Your Name_Client-Centric Consistency_Written Report.* You must submit this report in the designated submission space in the course. Learners may **not** email or use other means to submit this project deliverable for course team review and grading. Learners should review the academic integrity and plagiarism policies prior to beginning this project. Rubrics communicate specific criteria for evaluation. Prior to starting any graded coursework, you are expected to read through the rubric found on the project overview document, so you know how you will be assessed. You are encouraged to self-assess your work and make informed revisions before submitting. Engaging in this learning practice will support you in developing your highest quality deliverables. *Points may be deducted at the discretion of the course team for disorganized submissions that convolute the grading process.*

Problem Statement

This **Client-Centric Consistency Project** involves extending the gRPC project to address the consistency issues in a distributed banking system where a customer can interact with multiple branch replicas of the same account. Without the enforcement of consistency, operations like deposits, withdrawals, and querying an account balance across different branches can result in outdated or inconsistent results. In this project, we will remove the restriction from the earlier project, where a customer could only interact with their assigned branch. By allowing customers to use multiple branches and access their local versions of the data, we will be able to verify client-centric consistency and specifically enforce the "Read Your Writes" policy. This policy ensures that any write operation (e.g., deposit, withdrawal) performed by a customer is immediately visible to subsequent read operations (e.g., balance query) by the same customer, regardless of which replica is accessed.

Client-Centric Consistency Assumptions:

- Customers can perform deposits, withdrawals, and balance queries with any branch.
- Transactions in one branch will be propagated to the other branches.
- Each branch maintains a balance replica that should remain consistent with other branch replicas.
- Despite independent updates by a customer to a specific branch, every branch's replica should reflect all account changes made by other branches.

Goal

The goal of this project is to implement a distributed banking system based on the requirements outlined in the problem statement. Specifically, this system should support customer-to-branch communications, ensure consistent balance updates across branches, and maintain transactional integrity across distributed nodes. To ensure consistency, the Client-Centric Consistency model will be implemented, with a specific focus on enforcing the **Read Your Writes** policy.

Read Your Writes consistency ensures the following:

1. **Consistency Across Operations**: All operations (deposit, withdrawal, query) performed by a customer reflect the most recent writes by that customer, regardless of which branch replica is accessed.
2. **Tracking and Propagation of Writes**: Record updates made by customers and ensure that these updates are propagated to all replicas to maintain consistency.
3. **Seamless Customer Experience**: Enable customers to interact with any branch replica without encountering stale or inconsistent data.
4. **Validation Through Logging**: Provide detailed logs of all operations, including the tracking of account balances and their propagation, to validate the implementation of the Read Your Writes policy.

By achieving these objectives, the project ensures that the distributed banking system operates reliably, offering customers a consistent and accurate experience while addressing critical challenges in distributed data management. The diagram below illustrates the system design for communication between customers and branches:

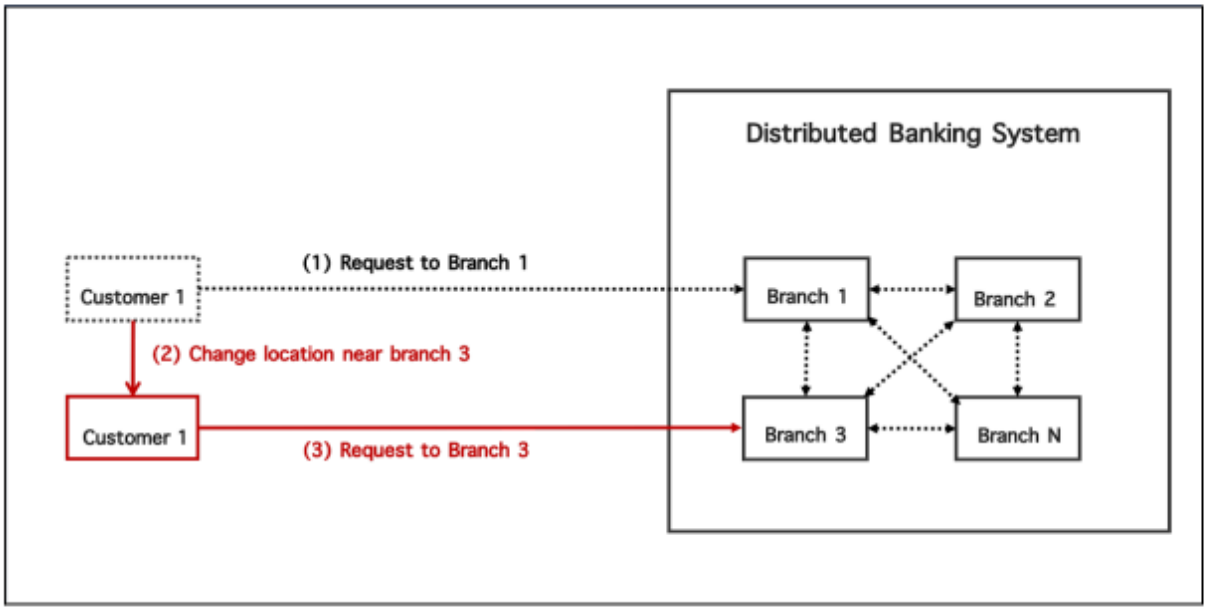


Diagram A: The customer changes the branches while submitting request to the Bank

Major Tasks

1. **Design Processes**: Design customer and branch processes using Python's multiprocessing module for parallelism.
2. **Branch Access Flexibility**: Allow customers to use any available branch.
3. **Branch Server Management**: Develop a process to initialize and manage branch servers capable of handling deposits, withdrawals, and balance queries.
4. **Input Parsing**: Parse an input JSON file for initializing branch balances and processing customer requests.
5. **Inter-Process Communication**: Enable customer requests (deposit, withdrawal, and query) through inter-process communication using gRPC.
6. **Communication Interfaces**: Define communication interfaces (e.g., Query, Deposit, Withdraw, and Propagate_Request) for interaction between customers and branches.
7. **Message Logging**: Log messages along with the branch ID for response messages.
8. **Data Consistency Verification**: Use response messages, along with the responding branch ID, to test data consistency and validate the Read Your Writes policy.

Setup

The setup for the project ensures a robust environment to support distributed communication, synchronization, and process management.

Development Environment:

1. **Python 3.12.3**: Serves as the primary language for implementing distributed processes and enabling gRPC communication.
2. **PyCharm Professional Edition**: An integrated development environment (IDE) used for writing, testing, and debugging code.
3. **Anaconda 2024.06**: Handles dependency management and provides isolated environments for project development.

Key Python Packages:

1. **Multiprocessing**: Enables concurrent processes for customers and branches to simulate real-world parallelism.
2. **Concurrent.futures**: Offers high-level APIs for executing tasks asynchronously.
3. **gRPC (grpcio==1.64.1)**: Facilitates remote procedure calls (RPC) between distributed processes.
4. **gRPC Tools (grpcio-tools==1.64.1)**: Generates Python code from .proto files, enabling structured communication between clients and servers.
5. **Protocol Buffers (protobuf==5.27.2)**: Ensures efficient communication by standardizing message serialization.
6. **sys and json**: Used for handling command-line arguments and parsing input/output data.

This setup ensures seamless integration of distributed processes, accurate message serialization, and synchronized communication. It provides the tools and frameworks required to build a scalable and reliable distributed system.

Implementation Processes

The system design leverages **gRPC** to ensure consistent and synchronized communication between clients (customers) and servers (branches). Each implementation step is carefully structured to maintain data integrity and ensure consistent balances across all branches.

1. Proto File Definition:

- The banks.proto file defines two message types (MsgRequest and MsgResponse) and a Branch service with two **RPC methods**: MsgDelivery and MsgPropagation.
- **MsgRequest**: Contains fields for transaction ID (id), operation type (interface), transaction amount (money), and branch ID (branch), establishing a standard format for request messages.
- **MsgResponse**: Contains fields for operation type (interface), transaction result (result), branch ID (branch), and balance (money), ensuring a consistent structure for response messages.
- This setup ensures reliable communication between clients and branches with well-defined messages for each transaction type.

2. Python Code Generation Using gRPC Tools:

- Using the grpc_tools.protoc command, the .proto file is compiled into Python files:

    - **banks_pb2.py**: Defines the structure of messages such as MsgRequest and MsgResponse.
    - **banks_pb2_grpc.py**: Implements the gRPC framework for the defined Branch service and its methods.
- These files provide the foundation for gRPC-based client-server interactions, enabling Python-based communication between clients and branches.

3. Server-Side Implementation (Branch):

- The Branch class (defined in branch.py) extends BranchServicer and implements the MsgDelivery and MsgPropagation methods.
    - **MsgDelivery**: Processes deposits, withdrawals, and balance queries. It updates the branch's balance, returns transaction results, and propagates updates to ensure consistency across balance replicas.
    - **MsgPropagation**: Synchronizes transactions for other branches by processing deposit or withdrawal operations to maintain consistency across replicas.
- **Additional Methods**:
    - **createStubs**: Establishes gRPC channels for inter-branch communication, allowing branches to propagate updates.
    - **ProcessRequest**: Handles transaction requests and triggers propagation for deposits and withdrawals, using Propagate_Request to ensure balance consistency.

4. Server Initialization and Startup (server.py):

- This file manages the setup and initialization of branch servers:
    - Reads branch information from the gprc_input.json file.
    - Uses createBranchProcesses to create and start branch servers as independent processes.
    - Each branch process invokes branch_serve, initializing gRPC channels and client stubs to enable branch communication and transaction handling.

5. Client-Side Implementation (Customer):

- The Customer class (in customer.py) defines how customers communicate with branches and manage transaction requests:
    - **get_stub**: Replaces createStubs to establish gRPC channels for inter-branch communication. Customers can interact with any branch, and the function dynamically creates stubs if they do not already exist. It retrieves the stub for the requested branch from the stubs dictionary.
    - **executeEvents**: Processes transactions (deposit, withdrawal, query) from the input JSON file, sends them to the branches, and logs the results in recvMsg.
- Customers use stubs to send structured requests to branches and store the branch responses for validation.

6. Client Initialization and Startup (client.py):

- This file initializes customer processes by:

- Reading customer requests from the gprc_input.json file.
- Using createCustomerProcesses to create customer instances and handle their requests.
- The serveCustomer function executes customer transactions, logs results, and generates an output JSON file for validation and result analysis.

Results

The results of the implementation are captured in the output.json file, which logs each customer's transaction request, the resulting balance, and the branch the customer interacted with. The JSON output provides a clear record of each request—whether it is a deposit, withdrawal, or query—along with the updated balance and branch ID.

The output file demonstrates the following sequence of events to validate data consistency in the **Client-Centric Consistency Model**, specifically verifying the implementation of the **Read Your Writes** policy:

1. Query a branch for its balance.
2. Perform a deposit or withdrawal (Write) to update the balance.
3. Query the branch where the write occurred (Read) to validate the write.
4. Query another branch to check the balance (Read). This step verifies data consistency for the write operation under the **Read Your Writes** policy.

These steps are repeated multiple times, with logs generated to verify adherence to the **Client-Centric Consistency Model** and the **Read Your Writes** policy. Below is a sample from the JSON output generated after the processes were completed:


```
[
   {
      "id": 1,
      "recv": [
         {
            "interface": "query",
            "branch": 1,
            "balance": 0
         }
      ]
   },
   {
      "id": 1,
      "recv": [
         {
            "interface": "deposit",
            "branch": 1,
            "result": "success"
```

```
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "query",
            "branch": 1,
            "balance": 10
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "query",
            "branch": 2,
            "balance": 10
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "deposit",
            "branch": 2,
            "result": "success"
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "query",
            "branch": 2,
            "balance": 20
        }
```

```
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "query",
            "branch": 3,
            "balance": 20
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "deposit",
            "branch": 3,
            "result": "success"
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "query",
            "branch": 3,
            "balance": 30
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "query",
            "branch": 4,
            "balance": 30
        }
    ]
```

```
},
{
    "id": 1,
    "recv": [
        {
            "interface": "deposit",
            "branch": 4,
            "result": "success"
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "query",
            "branch": 4,
            "balance": 40
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "query",
            "branch": 5,
            "balance": 40
        }
    ]
},
{
    "id": 1,
    "recv": [
        {
            "interface": "deposit",
            "branch": 5,
            "result": "success"
        }
    ]
},
```

```
    {
        "id": 1,
        "recv": [
            {
                "interface": "query",
                "branch": 5,
                "balance": 50
            }
        ]
    },
    {
        "id": 1,
        "recv": [
            {
                "interface": "query",
                "branch": 6,
                "balance": 50
            }
        ]
    }
]
```


Summary

The output.json file confirms the successful implementation of the **Client-Centric Consistency Model** and the **Read Your Writes** policy. The distributed system consistently handles transactions, ensuring that all operations adhere to the defined consistency guarantees. The logs clearly illustrate that the system performs correctly across various transaction scenarios, demonstrating a fully functional, consistent, and reliable distributed banking application.